چهار مکانیسم: سمافور، مانیتور، دکر و پترسون مسئلهٔ خوانندگان و نویسندگان با

می‌شود، آنچه روی نخ‌های واقعی اندازه‌گیری می‌شود، و جاهایی که این دو هم‌رأی نیستند آنچه دربارهٔ همهٔ درهم‌روی‌ها اثبات

عنوانمسئلهٔ خوانندگان و نویسندگان با چهار مکانیسم
زیرعنوانسمافور، مانیتور، دکر و پترسون؛ اثبات و اندازه‌گیری
درسسیستم‌عامل پیشرفته
تمرین۷ — خوانندگان و نویسندگان
روشبررسی کامل مدل به همراه سنجش‌گر نخ‌محور
تعداد راه‌حل‌های modelled17 پروتکل با ۲ تا ۳ فرایند
تعداد قفل‌های سنجش‌شده15 قفل روی نخ‌های واقعی
خروجی تحویلیمقالهٔ فارسی و انگلیسی (Word و PDF)، شکل‌ها، کد و آزمون‌ها
مخزنgithub.com/kaspian021/Multilevel-Queue-Scheduling
شاخهExercise-7-Readers-Writers-Comparison

فهرست مطالب

چکیده و واژگان کلیدی	3
۱. صورت مسئله، و اینکه چرا چهار پاسخ دارد	4
۲. دو ابزار: یک اثبات و یک اندازه‌گیری	5
۲.۲ سنجش‌گر نخ‌محور	6
۲.۳ اشکال، و رفع آن	7
۳. راه‌حل‌ها، به‌شکلی که بررسی‌گر می‌پیماید	8
۳.۲ مانیتور	9
۳.۳ دکر	10
۳.۴ پترسون	11
۴. آنچه اثبات می‌شود	12
۴.۱ خواندنِ جدول	14
۴.۲ هر اثبات چقدر بزرگ بود	15
۴.۳ یک گواهِ نقض چه شکلی است	16
۴.۴ گرسنگی، به‌شکل یک دور منصفانه	17
۵. دکر و پترسون: درست، سریع، و فقط دو نفر	18
۶. این قفل‌ها چقدر خرج دارند	19
۶.۱ صفِ بدون‌جلوزدن چقدر خرج دارد	20
۷. ویژگی‌ای که مسئله برای آن وجود دارد	21
۸. گرسنگی، اندازه‌گیری‌شده	22
۸. اعدادِ گرسنگی	23
۸.۱ خواندنِ عددهای گرسنگی	24
۸.۲ تصویر آینه‌ای، و جایی که اندازه‌گیری دست برمی‌دارد	25
۹. آیا افزودن خواننده کمک می‌کند؟	26
۱۰. کدام را به کار ببریم	27
۱۱. روش، خروجی‌ها و محدودیت‌ها	28

چکیده و واژگان کلیدی

چکیده

دوم اندازه‌گیری است: یک سنجش‌گر نخ‌محور هر قفل را زیر بار می‌برد و ناوردایی را در تک‌تک ورودها بررسی می‌کند. که اصلن مسئله برای آن وجود دارد — اینکه آیا دو خواننده می‌توانند هم‌زمان داخل باشند — رأی می‌دهد. پرسش دست‌یافتنیِ یک پروتکل دو یا سه‌فرایندی را می‌پیماید و دربارهٔ ایمنی، نبود بن‌بست، نبود گرسنگی، و البته ویژگی‌ای پرسش جداگانه از هر کدام پرسیده شده است. پرسش اول اثباتی است: یک بررسی‌گر کامل مدل، همهٔ درهم‌روی‌های با متغیرهای شرط، و دو راه‌حل نرم‌افزاری دوفرایندیِ دکر و پترسون. در این تمرین هر چهار تا ساخته شده و سپس دو چهار مکانیسم کلاسیک معمولاً به‌عنوان پاسخ عرضه می‌شود و این چهار تا هم‌ارز نیستند: سمافور شمارنده، مانیتور قفلی می‌خواهد که نویسنده را از همه جدا کند و در عین حال بگذارد هر تعداد خواننده با هم در بخش بحرانی باشند. مسئلهٔ خوانندگان و نویسندگان

همین است: الگوی خوانندگان و نویسندگان را میان دو خواننده بالا می‌برند و نمی‌توانند آن را به نفر سوم برسانند. به‌عنوان قفل درست‌اند و گرسنگی ندارند، و برای دقیقن دو شرکت‌کننده نوشته شده‌اند که تمام محدودیتشان همهٔ شرکت‌کنندگان گرسنه می‌مانند. اگر همین نخ بیدارشده را در صف ورودی بگذاریم، رأی برمی‌گردد. دکر و پترسون باید دوباره برای قفل مانیتور رقابت کند، انتظار را کران‌دار نمی‌کند: بررسی‌گر یک دور منصفانه پیدا می‌کند که در آن و بعد همان قفل شود. مانیتور مسا ایمن است و خوانش هم‌زمان را می‌پذیرد، اما چون نخی که اعلان دریافت کرده — و در بقیهٔ آزمون‌ها تقریبن همهٔ ترافیک از آنِ او بود؛ نتیجه به این بستگی دارد که چه کسی مسابقهٔ آغازین را ببرد در 6 آزمون از 9 آزمونِ نیم‌ثانیه‌ای با شش خواننده و یک نویسنده، سهم نویسنده از ترافیک کمتر از یک درصد بود جالب کار است. اولویت خوانندگان ایمن است، بن‌بست ندارد و سریع است، اما نویسنده را کاملن گرسنه می‌گذارد: این دو پاسخ همیشه هم‌رأی نیستند و همین ناهم‌رأیی‌ها بخش

واژگان کلیدی

انحصار متقابل، سمافور، مانیتور، معناشناسی مسا، الگوریتم دکر، الگوریتم پترسون، گرسنگی، بررسی مدل، انصاف مسئلهٔ خوانندگان و نویسندگان،

> **نکته:** همهٔ عددهای این مقاله را `exercise-7/code/run_experiments.py` مقاله هنگام ساخت آن‌ها را از `output/experiments.json` تولید می‌کند نشده است، پس مقاله نمی‌تواند از کدی که آن را ساخته فاصله بگیرد. می‌خواند. هیچ عددی با دست تایپ

۱. صورت مسئله، و اینکه چرا چهار پاسخ دارد

چه چیزی باید برقرار بماند

باید تنها باشد. سه ویژگی از اینجا می‌آید که سومی همان چیزی است که مسئله را به درد تمرین خوردن می‌اندازد: از یک شیء مشترک محافظت می‌کند. هر تعداد خواننده می‌توانند هم‌زمان آن را در دست داشته باشند؛ نویسنده قفل خوانندگان و نویسندگان

- **ایمنی**— هرگز دو نویسنده با هم، و هرگز یک نویسنده همراه کس دیگری داخل نباشد.
- **زنده‌بودن**نبود بن‌بست (همیشه هنوز کسی می‌تواند وارد شود) و نبود گرسنگی (همه سرانجام وارد می‌شوند).
— هر فرایندی که درخواست کند، سرانجام وارد شود. این به دو بخش تقسیم می‌شود:
- **هم‌زمانی**برآورده می‌کند و باز در این سومی شکست می‌خورد، که تمام دلیل وجود این مسئله همین است.
— واقعاً دو خواننده بتوانند با هم داخل باشند. یک قفل ساده دو ویژگی اول را بی‌نقص

چرا این چهار مکانیسم هم‌ارز نیستند

در هزینه، و در تضمینی که دربارهٔ انتظار می‌دهند فرق دارند — و همین چیزهایی است که ادامهٔ مقاله اندازه می‌گیرد. چیزی جز پرچم‌های مشترک نوشته شده‌اند، و هر دو برای دقیقن دو فرایند نوشته شده‌اند. این چهار تا در توان بیان، می‌ماند قفل را آزاد می‌گذارد. الگوریتم‌های دکر و پترسون راه‌حل‌های نرم‌افزاریِ محض برای انحصار متقابل‌اند که با سمافور یک شمارنده با صف است. مانیتور یک قفل به همراه متغیرهای شرط است، و نخی که منتظر

نکته: دو شرکت‌کننده می‌توانند انجام دهند و دومی را اصلاً نمی‌توانند، و این تمایز در بخش ۵ اثبات می‌شود نه ادعا. به‌عنوان «قفل شمارنده» درون یک راه‌حل خوانندگان و نویسندگان، و به‌عنوان کل راه‌حل. دکر و پترسون اولی را برای در سراسر مقاله این چهار مکانیسم در دو سطح مقایسه می‌شوند:

۲. دو ابزار: یک اثبات و یک اندازه‌گیری

۲.۱ بررسی‌گر مدل

`code/rw_model.py` ندارد و بن‌بست به شکل حالتی دیده می‌شود که از آن به بعد هیچ ورودی در دسترس نیست. جلو برود یک خودحلقه تولید می‌کند که پیماینده آن را هرس می‌کند، بنابراین فرایند مسدود ساده‌جات جانشینی برابر است با (شمارنده‌های مشترک، سمافورها، شمارنده‌های برنامه، ثبات‌های خصوصی). فرایندی که نمی‌تواند یک پروتکل را برمی‌شمارد. حافظه پیاپی‌سازگار است، هر خواندن و نوشتن یک واژه اتمی است، و حالت سراسری فضای حالت دست‌یافتنی

- **ایمنی** روی تک‌تک حالت‌ها، همان‌طور که به آن‌ها می‌رسیم، بررسی می‌شود.
- **نبود بن‌بست** هست که کسی در حال تلاش باشد، کسی داخل نباشد، و هیچ ورودی دیگر در دسترس نباشد؟ دست‌یافتنی معکوس است: آیا حالتی
- **نبود گرسنگی** جست‌وجوی یک دور منصفانه است که در پایین توضیح داده می‌شود.
- **خوانش هم‌زمان** ویژگی مثبت است: آیا حالتی در دسترس هست که در آن دو خواننده با هم داخل باشند؟

انصاف همان بخشی است که خراب‌کردنش آسان است

رخ داد و هر سه گرفته شد، و نام بردن هر کدام ارزش دارد چون هر کدام یک پاسخ مطمئنِ غلط تولید کرده بود: را ثابت نمی‌کند: این زمان‌بندِ غیرمنصفانه است، نه قفل شکسته. هنگام ساخت این بررسی‌گر سه اشتباه جداگانه دوری که در آن قربانی اصلاً زمان‌بندی نشود چیزی

- انصاف فقط برای **قربانی** ابد نگه دارد، و قربانی را گرسنه نشان بدهد. انصاف باید از همهٔ فرایندها خواسته شود. اجازه می‌دهد فرایند سومی روی گامِ فعالی که هرگز برنمی‌دارد بنشیند، قفلی را تا
- به‌سوی بخش باقیمانده‌اش، دقیقن همان گذاری را حذف می‌کند که ثابت می‌کند او زمان‌بندی شده بوده است. هرس کردن گامِ خودِ قربانی
- یک **مسیر** بار از قربانی جلو می‌زند چیزی را ثابت نمی‌کند؛ فقط یک دور بسته است که می‌تواند تا ابد تکرار شود. که دو

> **نکته:** جزئی از خودِ دور باشد. اگر فرایند در جایی از دور مسدود باشد، تعهدی وجود ندارد و دور گرسنگیِ واقعی است. یک دور برای یک فرایند وقتی منصفانه است که در هر حالتِ آن دور که آن فرایند هر گامِ فعالی دارد، یکی از همان گام‌ها قاعده‌ای که بررسی‌گر سرانجام پیاده می‌کند این است:

۲.۲ سنجش‌گر نخ‌محور

چه چیزی را اجرا می‌کند

`code/rw_sync.py`یک قفل واقعی پیاده می‌کند و زیر بار اجرا می‌کند. دو تصمیم طراحی در اینجا اتفاقی نیست. هر راه‌حل را به‌شکل

- **سمافور اکیداً FIFO است.**می‌گیرد و به ترتیب بلیت خدمت می‌گیرد، پس تازه‌وارد نمی‌تواند از منتظر جلو بزند. نخ هنگام ورود یک بلیت`threading.Semaphore`اندازه می‌گیرد تا قیمت این صف یک عدد باشد نه یک ادعا. این‌طور رفتار نمی‌کند، و استفاده از آن بی‌سروصدا اثبات‌های انصافِ پایین را باطل می‌کرد. مقاله هر دو را
- **ناوردایی در هر ورود بررسی می‌شود، نه در پایان.**تعداد خوانندگانی که هم‌زمان داخل بوده‌اند چند تا بوده است. شود ردی نمی‌گذارد. سنجش‌گر تخلف را همان لحظه که رخ می‌دهد ثبت می‌کند، و نیز ثبت می‌کند که بیشترین قفلی که برای یک میکروثانیه نویسنده را کنار یک خواننده راه بدهد، در هیچ شمارنده‌ای که بعداً خوانده

آنچه سنجش‌گر گرفت

اگر گرفتن`wrt`کافی بزرگ است که مرتب زده شود. مدل از ابتدا درست بود، و همین برهانِ داشتنِ هر دو ابزار است. سنجش‌گر این را در کمتر از یک ثانیه بار پیدا کرد، روی گونه‌ای که سمافور بدون صف دارد و پنجره در آن به اندازهٔ — پنجره‌ای باز می‌شود که در آن خوانندهٔ دوم شمارنده را بالا برده اما خوانندهٔ اول هنوز قفل نویسنده را نگرفته است. بیرون از قفلِ شمارنده نوشته شود — روشِ بدیهی نوشتن، و روشی که اول نوشته شده بود

> **نکته:**که اتفاقاً انتخاب نکرده چیزی نمی‌گوید. با هم، اشکالی را گرفتند که هر کدام به‌تنهایی از دست می‌دادند. دربارهٔ پانزده قفل که یک ثانیه از ساعتِ دیواری اجرا می‌شوند چیزی نمی‌گوید. سنجش‌گر هم دربارهٔ درهم‌روی‌ای از این دو ابزار زائد نیست. بررسی‌گر مدل دو یا سه فرایند را برای مشتی گام می‌پیماید و دربارهٔ همهٔ آن‌ها حکم می‌دهد؛ هیچ‌کدام

۲.۳ اشکال، و رفع آن

```
# wrong: the count is raised, the mutex is dropped, and only
# then is wrt taken -- in that window a writer can be inside
# together with a reader
with self._mutex:
    self._readers += 1
    first = self._readers == 1
if first:
    self._wrt.acquire()          # too late

# right: wrt is taken while the count is still protected
with self._mutex:
    self._readers += 1
    if self._readers == 1:
        self._wrt.acquire()
```

اولویت خوانندگان، پیش و پس از رفع اشکال

چرا این پنجره وجود دارد

نویسنده درست به همان مدت که هر خواننده‌ای داخل است در دست باشد». میان آزاد کردن قفل شمارنده و گرفتن ناوردایی این نیست که «شمارنده درست است»؛ این است که «سمافورwrtشمارنده را دو می‌بیند و بی‌آنکه اصلاً به خواننده‌ای در راه است اما هنوز هیچ چیز جلوی نویسنده را نگرفته است. خوانندهٔ دومی که در این پنجره برسد ، شمارنده از قبل می‌گویدwrtبحرانی می‌شود، و در نتیجه قفل نویسنده دیگر معنایی را که پروتکل فرض می‌کند ندارد. دست بزند مستقیم وارد بخش

:نکتهاشتباهِ نظیر در سمتِ آزادسازی قرینه است و به همان اندازه آسان: پس دادنِ wrtشمارنده هنوز بالای صفر است. بعد از رها کردن قفل شمارنده، نویسنده را در حالی راه می‌دهد که

۳. راه‌حل‌ها، به‌شکلی که بررسی‌گر می‌پیماید

فهرستِ زیر برنامهٔ واقعیِ مدل‌شده است، یک خط به ازای هر گام اتمی، که هنگام ساخت از هر`code/rw_model.py`یک نسخهٔ شبه‌کدِ مرتب، همان گام‌هایی را پنهان می‌کند که راه‌حل‌ها در آن‌ها فرق دارند. همین گام‌ها دقیقن همان چیزی است که رأی‌های بخش ۴ دربارهٔ آن است، بنابراین سطح جزئیات خودِ مطلب است: استخراج می‌شود. درهم‌رویِ

۳.۱ اولویت خوانندگان

```
 0  p(mutex)
 1  rd = rd + 1
 2  r0 = rd
 3  r0 = r0 - 1
 4  if r0 != 0 goto 6
 5  p(wrt)
 6  v(mutex)
 7  enter
 8  exit
 9  p(mutex)
10  rd = rd - 1
11  r0 = rd
12  if r0 != 0 goto 14
13  v(wrt)
14  v(mutex)
15  remainder; goto 0
```

برنامهٔ خواننده در `rw_readers_pref`

`mutex` از شمارنده محافظت می‌کند و `wrt` نویسندگان را بیرون نگه می‌دارد. نخستین خواننده `wrt`توجه کنید که را می‌گیرد، واپسین آن را پس می‌دهد، و هر که میان این دو است ساده وارد می‌شود.`wrt`گرفته می‌شود، در حالی که در گام ۵`mutex`هنوز در دست است — همان جزئیاتی که بخش ۲.۳ دربارهٔ آن است.

۳.۲ مانیتور

```
 0  p(m)
 1  r0 = wactive
 2  if r0 == 0 goto 4
 3  wait(cv) -> 1
 4  r0 = rd
 5  if r0 == 0 goto 7
 6  wait(cv) -> 1
 7  wactive = 1
 8  v(m)
 9  enter
10  exit
11  p(m)
12  wactive = 0
13  notifyAll(cv)
14  v(m)
15  remainder; goto 0
```

برنامهٔ نویسنده در `rw_monitor_mesa`

مسا طلب می‌کند — نخی که اعلان گرفته باید دوباره به شرط نگاه کند، چون اعلان هیچ تعهدی با خود نیاورده بود. بیدار می‌کند. هر انتظار به آزمونِ دوباره در گام ۱ برمی‌گردد نه به ابتدای برنامه، که همان چیزی است که معناشناسیِ منتظر می‌ماند و هنگامی که یک فاز خوانش باز است دوباره منتظر می‌ماند؛ در خروج، پرچم را پاک می‌کند و همه را یک قفل، یک متغیر شرط. نویسنده هنگامی که نویسندهٔ دیگری فعال است

و گونهٔ منصفانهٔ آن

```
 0  p(m)
 1  r0 = wactive
 2  if r0 == 0 goto 4
 3  wait-queued(cv) -> 1
 4  r0 = rd
 5  if r0 == 0 goto 7
 6  wait-queued(cv) -> 1
 7  wactive = 1
 8  v(m)
 9  enter
10  exit
11  p(m)
12  wactive = 0
13  notifyAll+handoff(cv)
14  v(m)
15  remainder; goto 0
```

همان برنامه با `wait-queued` و `notifyAll+handoff`

دو دستور فرق می‌کند.`wait-queued`را به صف ورودی مانیتور می‌برد به‌جای اینکه بگذارد مسابقه بدهد، و نخِ بیدارشده`notifyAll+handoff`تفاوتِ میان «همه می‌توانند گرسنه بمانند» و «فقط نویسنده می‌تواند» است. این تحویل را همان لحظهٔ اعلان و اتمی انجام می‌دهد، نه اینکه نخ آگاه‌شده را منتظر زمان‌بندی بگذارد. این تمام

۳.۳ دکر

```
 0  flag[me] = 1
 1  r0 = flag[other]
 2  if r0 == 0 goto 11
 3  r0 = turn
 4  if r0 == me goto 1
 5  flag[me] = 0
 6  r0 = turn
 7  if r0 == me goto 9
 8  goto 6
 9  flag[me] = 1
10  goto 1
11  enter
12  exit
13  turn = other
14  flag[me] = 0
15  remainder; goto 0
```

الگوریتم دکر، آن‌گونه که مدل شده است

به داخل می‌پرید، که دقیقن عکسِ کاری است که دکر می‌کند، و بررسی‌گر هم بنا به وظیفه تخلفِ ایمنی گزارش داد. می‌برم. همین امتیاز دادن بخشی بود که درست کردنش یک اصلاح گرفت — نسخهٔ اول به محض دیدن نوبتِ خودش طرفِ دیگر علاقه‌مند است و نوبتِ من نیست، پرچمم را پایین می‌آورم، منتظر می‌مانم نوبت برگردد، و دوباره آن را بالا دو پرچم و یک نوبت. دکر امتیاز می‌دهد: وقتی

۳.۴ پترسون

```
 0  flag[me] = 1
 1  turn = other
 2  r0 = flag[other]
 3  if r0 == 0 goto 7
 4  r0 = turn
 5  if r0 == me goto 7
 6  goto 2
 7  enter
 8  exit
 9  flag[me] = 0
10  remainder; goto 0
```

الگوریتم پترسون، آن‌گونه که مدل شده است

– یکی برای نوشتن پرچمِ خودم و یکی برای خواندن پرچمِ دیگری – چون فرایند باید بداند کدام‌یک از آن دو است. پروتکل ورود چهار دستور است به‌جای یازده دستور. هر دو به دو شکلِ دستوری نیاز دارند که راه‌حل‌های دیگر ندارند پترسون حلقهٔ پرمشغله را حذف می‌کند: نوبت کار امتیازدادن را انجام می‌دهد، پس

و پترسون 267 حالت را. هیچ‌کدام دو خواننده را هم‌زمان راه نمی‌دهند، چون هیچ‌کدام دنبال این کار نیستند. دستور کمتر یعنی جای کمتر برای قطع شدن: دکر 443 حالت را می‌پیماید

۴. آنچه اثبات می‌شود

و آن هم به قصد: در نخستین گواه خود می‌ایستد، که برهانی است بر شکستِ ناوردایی، نه بررسیِ آنچه بعداً می‌افتد. هر سطر زیر کامل است: هیچ رأیی بر فضای حالتِ بریده‌شده استوار نیست. راه‌حلِ خراب تنها استثناست

راه‌حل	نقش‌ها	حالت‌ها	ایمنی	نبود بن‌بست	گرسنه می‌ماند	خوانش هم‌زمان
`semaphore_mutex`	RW	28	برقرار	بله	هیچ‌کس	خیر
`monitor_mesa_mutex`	RW	146	برقرار	بله	**همهٔ آن‌ها**	خیر
`monitor_fifo_mutex`	RW	122	برقرار	بله	هیچ‌کس	خیر
`dekker_mutex`	RW	443	برقرار	بله	هیچ‌کس	خیر
`peterson_mutex`	RW	267	برقرار	بله	هیچ‌کس	خیر
`rw_readers_pref`	RWR	1,797	برقرار	بله	**فرایند 1(W)**	بله
`rw_writers_pref`	RWR	6,876	برقرار	بله	هیچ‌کس	بله
`rw_writers_pref_2w`	RWW	5,210	برقرار	بله	**فرایند 0(R)**	خیر
`rw_fair`	RWR	2,715	برقرار	بله	هیچ‌کس	بله
`rw_barrier`	RWR	6,876	برقرار	بله	هیچ‌کس	بله
`rw_monitor_mesa`	RWR	3,512	برقرار	بله	**همهٔ آن‌ها**	بله
`rw_monitor_fifo`	RWR	2,756	برقرار	بله	**فرایند 1(W)**	بله
`rw_dekker`	RW	155	برقرار	بله	هیچ‌کس	خیر
`rw_dekker_2readers`	RR	2,430	برقرار	بله	هیچ‌کس	بله
`rw_peterson`	RW	116	برقرار	بله	هیچ‌کس	خیر
`rw_peterson_2readers`	RR	1,382	برقرار	بله	هیچ‌کس	بله
`rw_broken_count`	RWR	144	**نقض‌شده**	**خیر**	هیچ‌کس	خیر

جدول 1. بررسی کامل مدل، ۲ تا ۳ فرایند برای هر راه‌حل

Exhaustive model checking: every verdict, 2-3 processes

solution	roles	states	safe	deadlock free	starves	concurrent reads
sem mutex	RW	28	True	True	all	False
monitor mesa	RW	146	True	True	none	False
monitor fifo	RW	122	True	True	all	False
dekker	RW	443	True	True	all	False
peterson	RW	267	True	True	all	False
readers pref	RWR	1797	True	True	P1	True
writers pref (1w)	RWR	6876	True	True	all	True
writers pref (2w)	RWW	5210	True	True	P0	False
fair fifo	RWR	2715	True	True	all	True
writer barrier	RWR	6876	True	True	all	True
monitor mesa	RWR	3512	True	True	none	True
monitor fifo	RWR	2756	True	True	P1	True
dekker rw	RW	155	True	True	all	False
dekker rw (2r)	RR	2430	True	True	all	True
peterson rw	RW	116	True	True	all	False
peterson rw (2r)	RR	1382	True	True	all	True
broken (off by one)	RWR	144	False	False	all	False

شکل 2. همان رأی‌ها به‌شکل تصویر: سبز یعنی برقرار، قرمز یعنی نه، نارنجی یعنی کسی را گرسنه می‌گذارد

قفل است که کار قفل را می‌کند، یا یک راه‌حل خوانندگان و نویسندگان که بهای انصاف را به ارزی یا ارز دیگر می‌پردازد. دو دستور یک برنامه‌اند، و یکی همهٔ شرکت‌کنندگان را گرسنه می‌گذارد و دیگری فقط نویسنده را. بقیهٔ جدول یا یک دو سطری که بار استدلال را می‌کشند مانیتور مسا و مانیتور با صف ورودیِ منصفانه‌اند. این دو جدایِ

۴.۱ خواندنِ جدول

هر راه‌حلِ انحصار متقابل دو ویژگی را می‌گذراند و در سومی شکست می‌خورد

را برآورده می‌کنند و نکته‌اش را کاملن از دست می‌دهند. ستون آخر همان چیزی است که مسئله برای آن وجود دارد. دوفرایندی، ایمن‌اند و بن‌بست ندارند، و هیچ‌کدام هرگز دو خواننده را هم‌زمان راه نمی‌دهند. آن‌ها حرفِ مسئله سمافور قفل، مانیتور قفل، و هر دو الگوریتم کلاسیک

را گرسنه می‌گذارد؛ اولویت نویسندگان برای گرسنه گذاشتنِ خواننده به دو نویسنده نیاز دارد اولویت خوانندگان نویسنده

`rw_readers_pref`برسد یکی کم و یکی زیاد کند. تصویر آینه‌ای به‌شکل آموزنده‌ای نامتقارن است. با یک نویسنده (دو خواننده می‌توانند شمارنده را تا ابد بالای صفر نگه دارند، بدین‌صورت که هر کدوم پیش از آن‌که دیگری به صفر ایمن است، بن‌بست ندارد و خوانش هم‌زمان را می‌پذیرد، و نویسنده هرگز وارد نمی‌شود:`rw_writers_pref`(هیچ‌کس گرسنه نمی‌ماند: در خواننده هر بار که نویسنده کارش تمام شود باز می‌شود. نویسندهٔ دوم را اضافه کنید)`rw_writers_pref_2w`«بار» است نه فقط ویژگیِ «الگوریتم»، و مدلی با یک نویسنده اصلاً نمی‌تواند آن را ببیند.) و در دیگر باز نمی‌شود، و این بار خواننده است که گرسنه می‌ماند. گرسنگی ویژگی

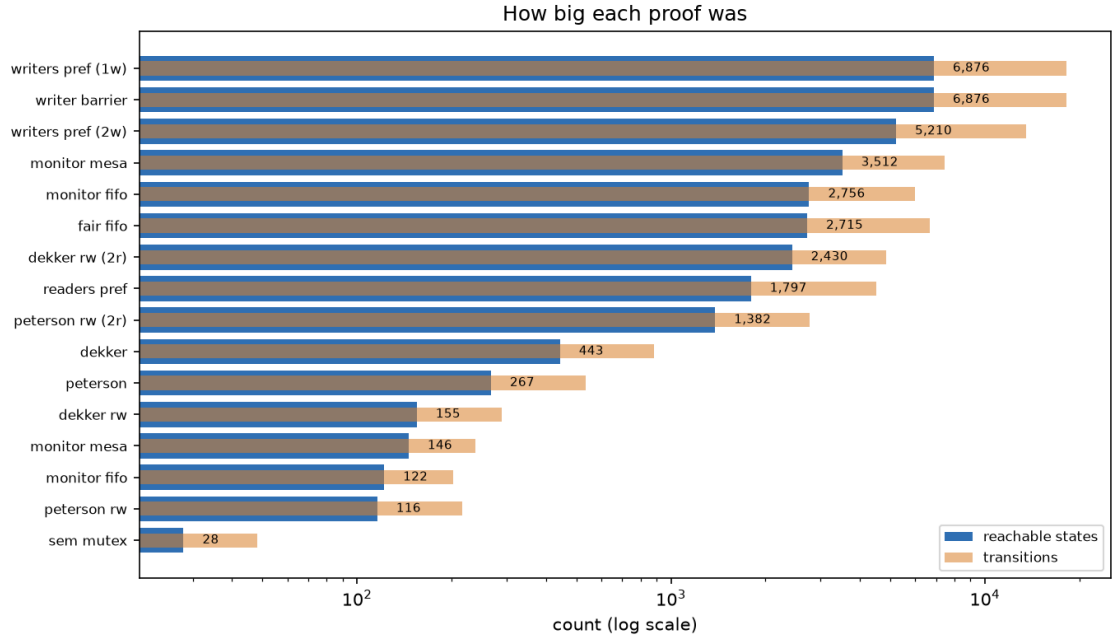
مانیتور مسا انتظار را کران‌دار نمی‌کند؛ صف ورودیِ منصفانه درستش می‌کند

`rw_monitor_mesa`را بگیرد، و تازه‌وارد می‌تواند زودتر آن را بردارد. جاوا همین‌طور رفتار می‌کند، و همین است که بماند. دلیلش همان یک جزئیاتی است که معناشناسی مسا باز می‌گذارد: نخی که اعلان گرفته باید دوباره قفل مانیتور ایمن است، بن‌بست ندارد و خوانش هم‌زمان را می‌پذیرد، و هر شرکت‌کننده‌ای می‌تواند گرسنه`notify`بگذارید — و هیچ تضمین انصافی با خود ندارد. یک چیز را عوض کنید — نخِ بیدارشده را به‌جای مسابقه دادن در صف ورودی `rw_monitor_fifo`اولویت خوانندگان منصف می‌شود: نویسنده می‌تواند گرسنه بماند، هیچ‌کسِ دیگر نمی‌تواند. دقیقن به اندازهٔ

> **نکته:**جدول است، چون دو مدلِ مانیتور در یک دستور فرق دارند. آنچه کم بود هرگز متغیر شرط نبود؛ صفِ ورودی بود. این روشن‌ترین نتیجهٔ

۴.۲ هر اثبات چقدر بزرگ بود

گذشت و تمام نشد، و با خطای ارزان‌تری عوض شد: یک خطای یک‌واحدی در شرط، که در 144 حالت فرو می‌پاشد. دارد. یک گونهٔ خراب که نخست امتحان شد — به‌روزرسانیِ ازدست‌رفته روی شمارندهٔ خواننده — از ۸۰۰ هزار حالت آزادی درهم‌روی است. راه‌حلِ سد و اولویت نویسندگان 6,876 حالت را می‌پیمایند؛ سمافور قفل به 28 حالت نیاز فضاهای حالت بیش از دو مرتبهٔ بزرگی با هم فرق دارند، و این اندازه معیار پیچیدگیِ پروتکل نیست — معیار



شکل 3. حالت‌های دست‌یافتنی و گذارها برای هر راه‌حل، مقیاس لگاریتمی

نکته: می‌توان فهمید بررسی‌گر اصلاً نگاه می‌کند: بررسی‌گری که تا به حال راه‌حلی را رد نکرده باشد، آزموده نشده است. مدل کردنِ یک راه‌حلِ خراب به‌قصد ارزش دارد. این تنها راهی است که

۴.۳ یک گواهِ نقض چه شکلی است

بررسی‌گر کوتاه‌ترین درهم‌روی‌ای را که ویژگی را می‌شکند چاپ می‌کند، و خواندنِ دو تا از آن‌ها به‌طور کامل ارزش دارد. رأی بدون گواه یک ادعاست.

ایمنی، با یک مقایسه شکسته

`rw_broken_count`دارد، و بررسی‌گر در 144 حالت به نویسنده‌ای می‌رسد که همراه یک خواننده داخل است: بررسی می‌کند آیا شمارنده صفر است به‌جای اینکه بررسی کند آیا یک است. این یک مقایسه با راه‌حل درست فاصله همان اولویت خوانندگان است با شرطی که یک واحد جابه‌جا شده است: نخستین خواننده

```
pcs=(0, 0, 0) {mutex=1q[], wrt=1q[]} shared=(0,)
P0(R): pcs=(1, 0, 0) {mutex=0q[], wrt=1q[]} shared=(0,)
P0(R): pcs=(2, 0, 0) {mutex=0q[], wrt=1q[]} shared=(1,)
P0(R): pcs=(3, 0, 0) {mutex=0q[], wrt=1q[]} shared=(1,)
P0(R): pcs=(5, 0, 0) {mutex=0q[], wrt=1q[]} shared=(1,)
P0(R): pcs=(6, 0, 0) {mutex=1q[], wrt=1q[]} shared=(1,)
P0(R): pcs=(7, 0, 0) {mutex=1q[], wrt=1q[]} shared=(1,) ENTER
P1(W): pcs=(7, 1, 0) {mutex=1q[], wrt=0q[]} shared=(1,)
P1(W): pcs=(7, 2, 0) {mutex=1q[], wrt=0q[]} shared=(1,) ENTER
```

گواهِ ایمنی برای `rw_broken_count`

فرایند ۰ شمارنده را به یک می‌رساند، چون شرط مقدار غلط را می‌آزماید ازwrtو وارد می‌شود. فرایند ۱ سپس می‌گذرد،wrtاست، چون هیچ‌کس آن را نگرفته بود — و او هم وارد می‌شود. دو سطر از ردّ پا، و ناوردایی رفته است. را می‌گیرد — که آزاد

۴.۴ گرسنگی، به‌شکل یک دور منصفانه

کل پروتکلش را دور می‌زند و وارد می‌شود؛ فرایند ۰ بیدار می‌شود، دوباره می‌آزماید، و باز بخش را اشغال‌شده می‌یابد. مسا همهٔ شرکت‌کنندگان را گرسنه می‌گذارد، و این همان دوری است که آن را برای فرایند ۰ ثابت می‌کند. فرایند ۲ مانیتور

```
P2(R): pcs=(3, 6, 13) {m=0q[], cv=2q[0, 1]} shared=(0, 0)
P2(R): pcs=(3, 6, 14) {m=1q[], cv=2q[0, 1]} shared=(0, 0)
P2(R): pcs=(3, 6, 0) {m=1q[], cv=2q[0, 1]} shared=(0, 0)
P2(R): pcs=(3, 6, 1) {m=0q[], cv=2q[0, 1]} shared=(0, 0)
P2(R): pcs=(3, 6, 2) {m=0q[], cv=2q[0, 1]} shared=(0, 0)
P2(R): pcs=(3, 6, 4) {m=0q[], cv=2q[0, 1]} shared=(0, 0)
P2(R): pcs=(3, 6, 5) {m=0q[], cv=2q[0, 1]} shared=(1, 0)
P2(R): pcs=(3, 6, 6) {m=1q[], cv=2q[0, 1]} shared=(1, 0)
P2(R): pcs=(3, 6, 7) {m=1q[], cv=2q[0, 1]} shared=(1, 0) ENTER
P2(R): pcs=(3, 6, 8) {m=1q[], cv=2q[0, 1]} shared=(1, 0)
P2(R): pcs=(3, 6, 9) {m=0q[], cv=2q[0, 1]} shared=(1, 0)
```

گواهِ گرسنگی برای `rw_monitor_mesa`، فرایند ۰

که پس از او رسیده است. تکرار این دور او را تا ابد منتظر نگه می‌دارد، که همان معنای «انتظار کران‌دار ندارد» است. نیست قربانی نادیده گرفته شود. فرایند ۰ اعلان می‌گیرد، بیدار می‌شود، و رقابت برای قفل مانیتور را به نخی می‌بازد هر فرایندی روی این دور یا زمان‌بندی می‌شود یا مسدود است، پس زمان‌بندی منصفانه است: اینجا قرار

نکته: همین دور در `rw_monitor_fifo` لحظهٔ اعلان در صف ورودی جا می‌گیرد. یک دستور، و دور ناپدید می‌شود. وجود ندارد، چون در آنجا نخ آگاه‌شده همان

۵. دکر و پترسون: درست، سریع، و فقط دو نفر

به‌عنوان قفل، درست‌اند

دکر 443 حالت و پترسون 267 حالت را می‌پیماید. پترسون کوتاه‌تر و اثباتش ارزان‌تر است، که با شهرتش می‌خواند.
تا پرچمِ خود را بنویسد و پرچمِ دیگری را بخواند — و هر دو ایمن، بدون بن‌بست و بدون گرسنگی از کار درآمده‌اند.
هر دو الگوریتم با توسعه‌های دستوریِ خودشان مدل شده‌اند — فرایند باید اندیسِ خودش را بداند

و می‌توانند الگو را بالا ببرند، میان دو شرکت‌کننده

خواننده» بگذارید، بگذارید نخستین خواننده سمافور نویسنده را بگیرد، و دو خواننده سر از داخلِ هم‌زمان درمی‌آورند.
که آیا می‌توان الگوی خوانندگان و نویسندگان را روی آن‌ها ساخت. می‌توان. قفل را به‌جای دور بخش، دور «شمارندهٔ
جالب این نیست که آیا دکر و پترسون به‌عنوان قفل کار می‌کنند — می‌کنند — بلکه این است
پرسش`rw_dekker_2readers` و `rw_peterson_2readers`را بدون هیچ سمافوری در مدل می‌پذیرند.
هر دو ایمن‌اند، بن‌بست ندارند، گرسنگی ندارند، و خوانش هم‌زمان

خوانش هم‌زمان	ایمنی	حالت‌ها	نقش‌ها	راه‌حل
خیر	برقرار	155	RW	`rw_dekker`
بله	برقرار	2,430	RR	`rw_dekker_2readers`
خیر	برقرار	116	RW	`rw_peterson`
بله	برقرار	1,382	RR	`rw_peterson_2readers`

جدول 4. الگوریتم‌های کلاسیک در حال بالا بردنِ الگوی خوانندگان

و همان‌جا متوقف می‌شوند

مانیتور مسئله را برای هر تعداد شرکت‌کننده بیان می‌کنند. دکر و پترسون انحصار متقابل را برای دو نفر بیان می‌کنند.
تعریفِ کاری است که برایش ساخته شده‌اند — اما همان پاسخی است که این تمرین از مقایسه می‌خواهد. سمافور و
قفل دور همه‌چیز، یکی‌یکی، و هم‌زمانی‌ای که مسئله برای آن وجود دارد ناپدید می‌شود. این نقدِ الگوریتم‌ها نیست —
نیست. اگر روی مسئلهٔ خوانندگان و نویسندگان با بیش از دو شرکت‌کننده به کار بروند، به یک قفل فرو می‌ریزند: یک
هر دو الگوریتم برای دقیقن دو فرایند نوشته شده‌اند. پرچمِ سومی و نوبتِ سومی در کار

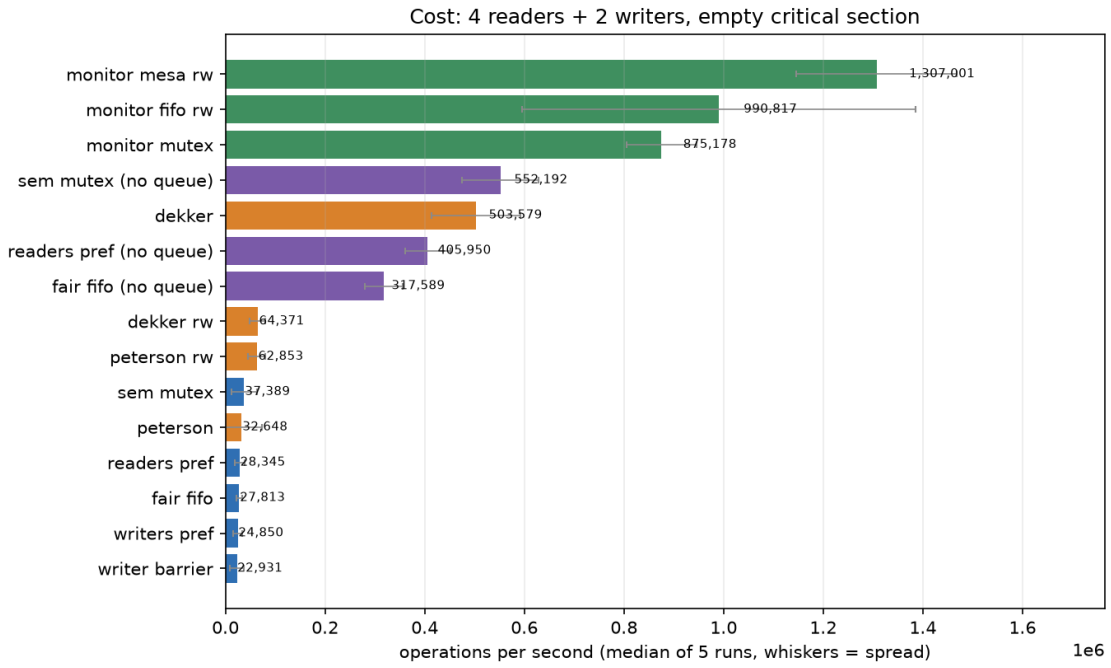
> **نکته:**اندازه‌گیری همین را از سوی دیگر می‌گوید: با یک خواننده و یک نویسنده، `dekker_rw` و `peterson_rw`باشند.
> ترافیک را تقریبن مساوی تقسیم می‌کنند، با سهمِ نوشتنِ 0.42 و 0.45. نفر سومی نیست که نسبت به او غیرمنصف

۶. این قفل‌ها چقدر خرج دارند

«موازی‌سازی»: نخ‌های CPython موازی اجرا نمی‌شوند، پس هیچ قفلی در اینجا با افزودن خواننده سریع‌تر نمی‌شود. تهی، 5 اجرا برای هر قفل، و میانه گزارش شده با پراکندگی در کنارش. این جدول را رتبه‌بندیِ «سربار» بخوانید نه چهار خواننده و دو نویسنده، بخش بحرانی

قفل	عملیات بر ثانیه (میانه)	پراکندگی	سهمِ نوشتن	خوانندگانِ داخل بیشترین
سمافور (قفل ساده)	37,389	25,678	0.228	1
سمافور، بدون صف	552,192	76,847	0.392	1
مانیتور (قفل)	875,178	69,875	0.329	1
دکر	503,579	89,154	0.497	1
پترسون	32,648	39,935	0.003	1
اولویت خوانندگان	28,345	10,301	0.000	1
خوانندگان، بدون صف اولویت	405,950	45,487	0.524	3
اولویت نویسندگان	24,850	8,598	0.193	2
منصفانه FIFO	27,813	5,278	0.226	2
بدون صف FIFO منصفانه،	317,589	38,001	0.638	3
سد نویسنده	22,931	13,590	0.199	1
مانیتور مسا	1,307,001	160,850	0.000	4
مانیتور با صف منصفانه	990,817	395,166	0.275	4
به‌عنوان قفل شمارنده دکر	64,371	15,558	0.435	1
به‌عنوان قفل شمارنده پترسون	62,853	16,929	0.379	1

جدول 5. توان عملیاتی، ۴ خواننده + ۲ نویسنده، بخش بحرانی تهی



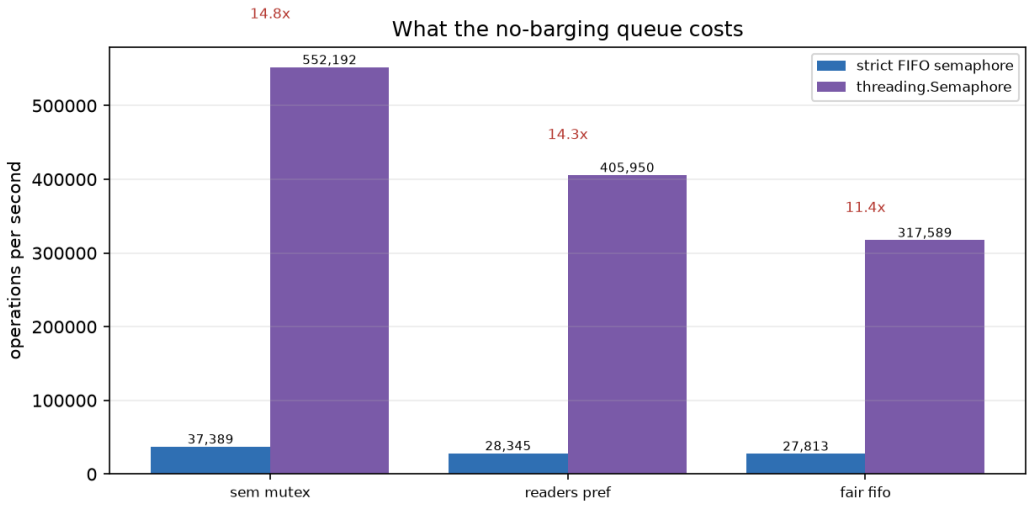
شکل 6. توان عملیاتی هر قفل؛ خطک‌ها پراکندگیِ اجراهای تکراری‌اند

۶.۱ صفِ بدون‌جلوزدن چقدر خرج دارد

سمافوری هست که نمی‌گذارد تازه‌وارد از منتظر جلو بزند. این ویژگی مجانی نیست، و این صورتحسابش است. اثبات‌های انصاف در بخش ۴ فرض می‌کنند

راه‌حل	سمافور FIFO اکید	threading.Semaphore	هزینهٔ صف
سمافور (قفل ساده)	37,389	552,192	14.8x
اولویت خوانندگان	28,345	405,950	14.3x
منصفانه FIFO	27,813	317,589	11.4x

جدول 7. هزینهٔ افزوده برای ویژگی‌ای که اثبات‌ها فرض می‌کنند



شکل 8. یک الگوریتم، یک بار، دو پیاده‌سازی سمافور

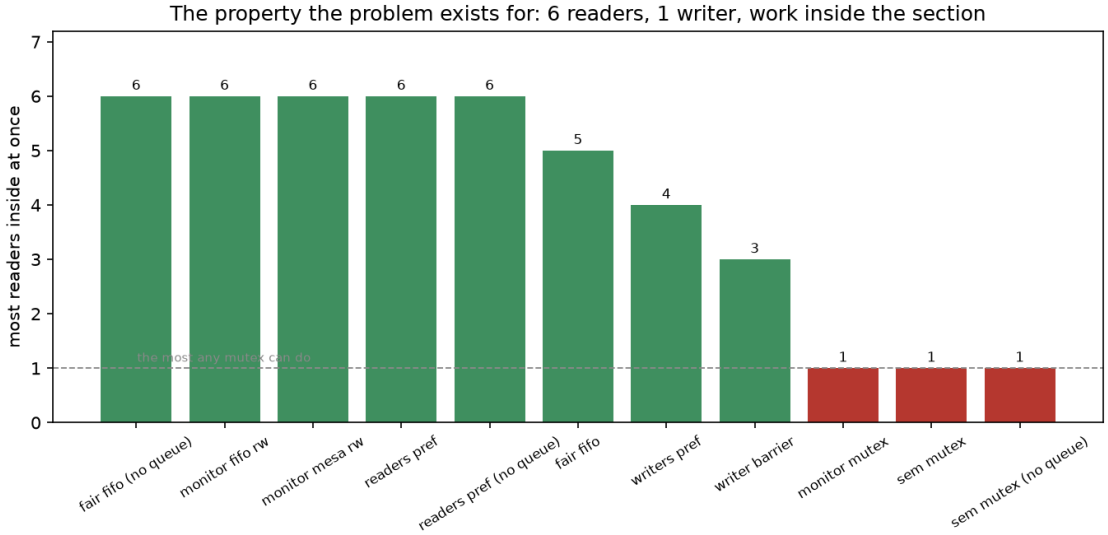
نکته: صف FIFO در این پیاده‌سازی است، نه ویژگی الگوریتم‌ها. درون یک خانواده مقایسه کنید، نه در عرض خانواده‌ها. همان خوانشِ درستِ جدولِ توان عملیاتی است: شکاف میان راه‌حل‌های مانیتور و راه‌حل‌های سمافور بیشترش هزینهٔ این

۷. ویژگی‌ای که مسئله برای آن وجود دارد

داخل دیده شده‌اند. یک قفل ساده نمی‌تواند بیش از یک بگیرد؛ هر چه یک گرفت، قفلی است با برچسبِ چیز دیگر. و کار واقعی درون بخش بحرانی تا اصلاً هم‌پوشانی ممکن باشد. عدد، بزرگ‌ترین تعداد خوانندگانی است که هم‌زمان شش خواننده، یک نویسنده،

قفل	بیشترین خوانندگانِ هم‌زمانِ داخل	سهم نوشتن
منصفانه، بدون صف FIFO	6	0.785
مانیتور با صف منصفانه	6	0.000
مانیتور مسا	6	0.000
اولویت خوانندگان	6	0.716
اولویت خوانندگان، بدون صف	6	0.000
منصفانه FIFO	5	0.104
اولویت نویسندگان	4	0.105
سد نویسنده	3	0.127
مانیتور (قفل)	1	0.000
سمافور (قفل ساده)	1	0.137
سمافور، بدون صف	1	0.000

جدول 9. خوانندگانِ هم‌زمان، ۶ خواننده + ۱ نویسنده، کار درون بخش بحرانی



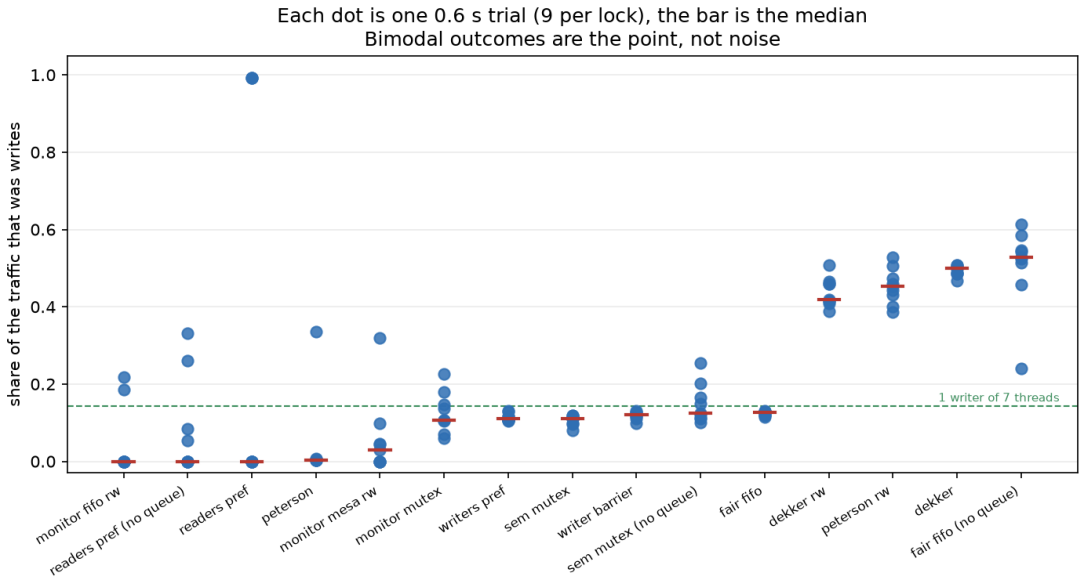
شکل 10. سبز یعنی دو خواننده یا بیشتر، قرمز یعنی هرگز

نکته: آنجا یعنی طراحی کار می‌کند، نه اینکه شکست خورده است. اینجا به 3 می‌رسد در برابر 6 برای اولویت خوانندگان. راه‌حلِ سد به‌قصد هم‌پوشانی را سقف‌دار می‌کند — سد برای همین است — پس عددِ کمتر در

۸. گرسنگی، اندازه‌گیری‌شده

شش خواننده در برابر یک نویسنده

آغازین را می‌برد و بعد برای بقیهٔ آزمون همان می‌ماند. میانهٔ دو مُد عددی است که هیچ‌کدام را توصیف نمی‌کند. ثبت می‌کند نه میانگینشان را، چون چند تا از این قفل‌ها دومُدی‌اند: نتیجه به این بستگی دارد که چه کسی مسابقهٔ این همان باری است که اولویت خوانندگان باید در آن شکست بخورد، و می‌خورد. شکل هر آزمون را جداگانه



شکل 11. هر نقطه یک آزمونِ ۰.۶ ثانیه‌ای است، میلهٔ قرمز میانه است، خط سبز یعنی یک نویسنده از هفت نخ

۸. اعدادِ گرسنگی

قفل	سهم نوشتن (میانه)	که نویسنده گرسنه ماند آزمون‌هایی	رشتهٔ خوانش طولانی‌ترین	رشتهٔ نوشتن طولانی‌ترین
مانیتور با صف منصفانه	0.000	7/9	1,094,802	55,557
خوانندگان، بدون صف اولویت	0.000	5/9	187,707	38,426
اولویت خوانندگان	0.000	6/9	10,895	19,850
پترسون	0.004	8/9	16,752	6,465
مانیتور مسا	0.030	4/9	1,252,307	33,857
مانیتور (قفل)	0.106	0/9	177,626	40,395
اولویت نویسندگان	0.111	0/9	3,458	1
سمافور (قفل ساده)	0.111	0/9	10,608	1
سد نویسنده	0.121	0/9	4,742	1
سمافور، بدون صف	0.124	0/9	107,483	20,546
منصفانه FIFO	0.128	0/9	2,222	1
به‌عنوان قفل شمارنده دکر	0.418	0/9	9,282	3,214
به‌عنوان قفل شمارنده پترسون	0.454	0/9	8,098	5,421
دکر	0.499	0/9	16,039	14,050
بدون صف FIFO منصفانه،	0.528	0/9	36,382	30,729

جدول 12. 9 آزمون برای هر قفل، ۶ خواننده + ۱ نویسنده

۸.۱ خواندنِ عددهای گرسنگی

اولویت خوانندگان، و هر دو مانیتور، نویسنده را گرسنه می‌گذارند

FIFO منصفانه، سد نویسنده، اولویت نویسندگان و هر دو قفل ساده در هیچ آزمونی کسی را گرسنه نگذاشتند. رشتهٔ ناشکستهٔ خوانش برای اولویت خوانندگان 10,895 عملیات است — فاز خوانشی که هرگز تمام نشد. گردونهٔ از 9 آزمون نویسنده را گرسنه گذاشت؛ مانیتور مسا در 4 آزمون و مانیتور با صفِ منصفانه در 7 آزمون. طولانی‌ترین اولویت خوانندگان در 6 آزمون

قفل چرخشیِ پترسون غافلگیرکننده است

انجام می‌دهد. شمارشِ چرخش‌ها گویاست — این دو قفل 17,548,790 چرخش در یک‌چهارم ثانیه ثبت کردند. زیر قفلِ مفسر سراسری، نخِ چرخان سراسر سهمیه‌اش را می‌سوزاند و نخِ دیگر هزاران عملیات در سهمیهٔ خودش مخالف است که هیچ ربطی به الگوریتم‌ها ندارد: نخی که می‌چرخد پردازنده را می‌گیرد و با آن هیچ کاری نمی‌کند. نویسنده‌غالب شد. هر دو به‌لحاظ اثباتی بدون گرسنگی‌اند، و بررسی‌گر مدل هم همین را می‌گوید. اندازه‌گیری به دلیلی الگوریتم پترسون در 8 آزمون از 9 آزمون نویسنده را گرسنه گذاشت، و دکر در 3 آزمون

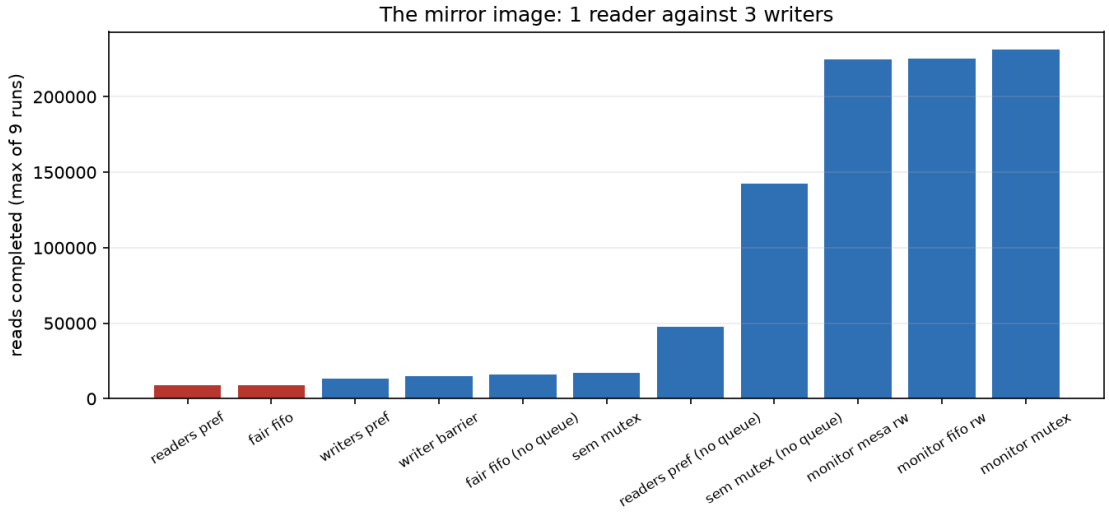
نکته: اندازه‌گیری دربارهٔ زمان‌بندِ CPython است، که تعهدی ندارد نسبت به نخی که می‌چرخد منصف باشد. است که در آن یک اثبات و یک اندازه‌گیری بی‌آنکه هیچ‌کدام غلط باشند مخالف‌اند. اثبات دربارهٔ زمان‌بندی‌های منصفانه این روشن‌ترین موردِ تمرین

۸.۲ تصویر آینه‌ای، و جایی که اندازه‌گیری دست برمی‌دارد

نویسندگان در خواننده را تا وقتی نویسنده‌ای منتظر است بسته نگه می‌دارد، پس سهم خواننده باید فرو بپاشد. یک خواننده در برابر چهار نویسنده. اولویت

قفل	سهم نوشتن (میانه)	طولانی‌ترین رشتهٔ نوشتن
منصفانه، بدون صف FIFO	0.959	122,344
اولویت خوانندگان، بدون صف	0.883	90,717
اولویت نویسندگان	0.845	19,135
مانیتور با صف منصفانه	0.806	319,199
سد نویسنده	0.802	21,323
مانیتور (قفل)	0.748	124,640
مانیتور مسا	0.719	150,744
اولویت خوانندگان	0.700	3,238
سمافور، بدون صف	0.678	78,106
منصفانه FIFO	0.666	2,484
سمافور (قفل ساده)	0.589	3,340

جدول 13. یک خواننده در برابر چهار نویسنده



شکل 14. خوانش‌های انجام‌شده زیر بار نویسنده‌سنگین

اندازه‌گیری اینجا چه می‌تواند نشان بدهد و چه نمی‌تواند

اولویت نویسندگان خواننده را تا ابد گرسنه می‌گذارد. اندازه‌گیری مکانیسم را تأیید می‌کند و نمی‌تواند اثر را جدا کند. اختیار انصافِ قفلِ خودِ مفسر، نه الگوریتم. گزارهٔ درست همان است که بررسی‌گر مدل عرضه می‌کند: با دو نویسنده، نویسندگان منصف است. سهمِ ترافیک در یک ثانیه در اختیار این است که چه کسی مسابقهٔ آغازین را برده، و در به‌طور قابل اتکایی از خط مبنا تشخیص‌پذیر نیست، و قفلِ مانیتور خودِ پایتون در این مقیاسِ زمانی «کمتر» از اولویت و ناشکسته تولید می‌کند، در حالی که راه‌حل‌های FIFO هر چند عملیات یک‌بار جابه‌جا می‌شوند، اما سهمِ نوشتنش نمی‌تواند آنچه را اثبات نشان می‌دهد نشان بدهد. اولویت نویسندگان واقعاً رشته‌های نوشتنِ بلند

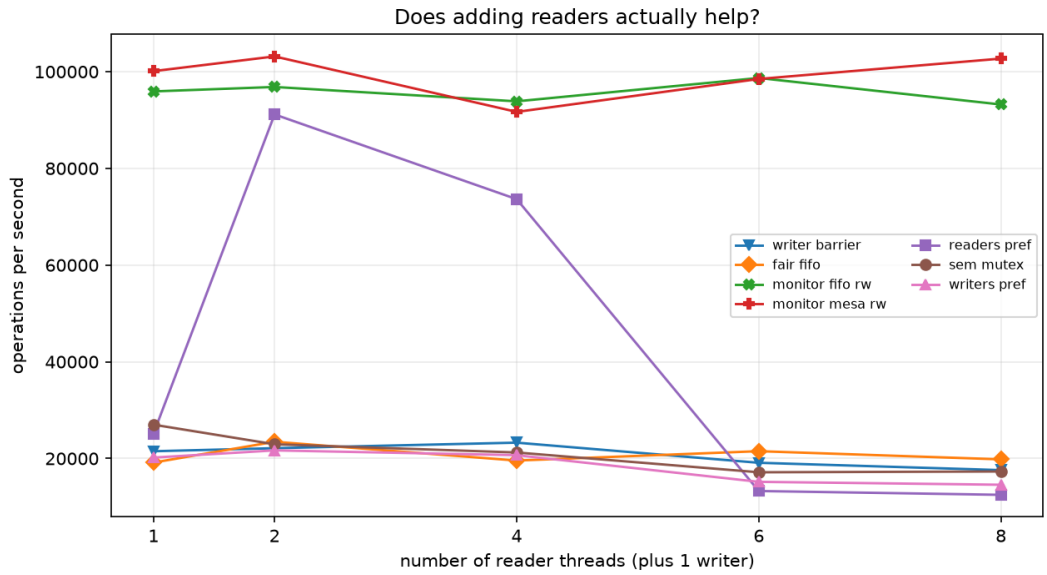
نکته:که همان‌طور گزارش شود. وسوسه این است که همان یک اجرایی نقل شود که پیش‌بینیِ نظریه را نشان می‌دهد. اندازه‌گیری‌ای که نمی‌تواند سیگنال را از نویز جدا کند باز هم ارزش گزارش شدن دارد، به‌شرطی

۹. آیا افزودن خواننده کمک می‌کند؟

قفل است، و نتیجهٔ درست این است که هیچ‌کدام از این راه‌حل‌ها در اینجا با تعداد خوانندگان مقیاس نمی‌گیرند. جلو بزند. روی CPython جلو نمی‌زند، چون نخ‌ها هیچ‌وقت هم‌زمان اجرا نمی‌شوند: آنچه نمودار نشان می‌دهد سربار روی ماشینی با موازی‌سازیِ واقعی، اینجا همان‌جایی است که قفل خوانندگان و نویسندگان باید از یک قفل ساده یک نویسنده، یک تا هشت خواننده، کار درون بخش بحرانی.

قفل	۱ خواننده	۲	۴	۶	۸
مانیتور مسا	100,105	103,152	91,676	98,488	102,698
با صف منصفانه مانیتور	95,914	96,835	93,861	98,692	93,208
منصفانه FIFO	19,149	23,452	19,566	21,505	19,815
سد نویسنده	21,492	22,123	23,269	19,106	17,588
سمافور (قفل ساده)	26,988	22,933	21,235	17,144	17,303
اولویت نویسندگان	20,182	21,668	20,683	15,166	14,562
اولویت خوانندگان	25,146	91,155	73,629	13,266	12,478

جدول 15. عملیات بر ثانیه در برابر تعداد نخ‌های خواننده، به همراه یک نویسنده



شکل 16. توان عملیاتی در برابر تعداد خوانندگان

نکته: همان لحظه‌ای که دو خواننده می‌توانند هم‌پوشانی کنند. این ادامه پیدا نمی‌کند، چون گلوگاه مفسر است نه قفل. واقعیِ جدول، رفتنِ اولویت خوانندگان از 25,146 عملیات بر ثانیه با یک خواننده به 91,155 با دو خواننده است — تنها حرکتِ

۱۰. کدام را به کار ببریم

چون	به کار ببرید	اگر نیاز دارید
نویسنده می‌تواند گرسنه بماند خانوادهٔ سمافور و ساده‌ترینشان؛ سریع‌ترین	اولویت خوانندگان	شرکت‌کننده، و تضمین انصاف لازم نیست هر تعداد
را در سراسر انتظارش نگه می‌دارد گرسنگی نداشت؛ نویسنده سمافوری سمافوری در اینجا که در هیچ آزمونی تنها راه‌حل	گردونهٔ FIFO منصفانه	هیچ‌کس گرسنه نماند
سقف‌دار می‌کند تا نویسندگان قفل نشوند متقارن؛ هم‌پوشانی خواننده را به‌قصد دو دروازهٔ	سد نویسنده	فازهای کران‌دار
صف ورودی یک بهبودِ اختیاری نیست مسا به‌تنهایی انتظار را کران‌دار نمی‌کند؛ معناشناسی	مانیتور با صف ورودیِ منصفانه	مانیتور
از دکر، و هر دو درست و بدون گرسنگی‌اند کوتاه‌تر و ارزان‌تر برای اثبات	پترسون	شرکت‌کننده، بدون پشتیبانی سیستم‌عامل دقیقن دو
فراتر از آن به یک قفل تنزل می‌کنند هر دو برای دو نفر نوشته شده‌اند؛	نه دکر، نه پترسون	بیش از دو شرکت‌کننده

جدول 17. آنچه رأی‌ها پشتیبانی می‌کنند

تنها درسی که تعمیم پیدا می‌کند

را گرسنه می‌گذارد و مانیتوری که فقط نویسنده را، یک دستور است دربارهٔ اینکه نخ بیدارشده کجا منتظر می‌ماند. ویژگیِ صف است. سمافور صف دارد و می‌توان روی آن حساب کرد. مانیتور ندارد، و تفاوتِ میان مانیتوری که همه انصاف ویژگیِ ابزار پایه نیست،

۱۱. روش، خروجی‌ها و محدودیت‌ها

چه چیزی روی این شاخه است

چیست	پرونده
بررسی‌گر کامل مدل، ۱۷ پروتکل	`code/rw_model.py`
پیاده‌سازی‌های نخ‌محور و سنجش‌گر	`code/rw_sync.py`
اجرای همهٔ آزمایش‌ها و نوشتنِ `output/experiments.json`	`code/run_experiments.py`
بازسازی همهٔ شکل‌ها	`code/make_diagrams.py`
۱۰۲ آزمون	`tests/test_exercise7.py`
هر عددِ این مقاله	`output/experiments.json`
۹ شکل، همه بازساختنی	`diagrams/`
این مقاله، فارسی	`docs/REPORT_FA.docx / .pdf`
این مقاله، انگلیسی	`docs/REPORT_EN.docx / .pdf`

جدول 18. پرونده‌های منتشرشده روی شاخهٔ تمرین ۷

محدودیت‌ها، صریح

- مدل دو یا سه فرایند را پوشش می‌دهد. ویژگی‌های پروتکل را اثبات می‌کند، نه ویژگی‌های هر تعداد نخ را. بررسی‌گر
- CPython اجرا می‌شوند، جایی که نخ‌ها موازی اجرا نمی‌شوند؛ عددهای توان عملیاتی سربارند، نه مقیاس‌پذیری. اندازه‌گیری‌ها روی
- FIFO اکید در این تمرین پیاده‌سازیِ ساده‌ای است به‌قصد، و تقریبن یک مرتبهٔ بزرگی از سمافور`threading.Semaphore`کندتر است. بخش ۶.۱ قیمتش را می‌دهد.
- نتیجه‌های گرسنگی در مقیاسِ نیم‌ثانیه برای چند قفل دومُدی است؛ هر آزمون گزارش شده نه میانگینشان.
- در اینجا نمونه‌برداری نشده است. بررسیِ مدل کامل است و سنجش‌گر ناوردایی را در هر ورود بررسی می‌کند. هیچ چیز

۷ — خوانندگان و نویسندگان	تمرین
Exercise-7-Readers-Writers-Comparison	شاخه
17	راه‌حل‌های مدل‌شده
15	قفل‌های سنجش‌شده
۱۰۲	آزمون‌های گذران
3.11.2	پایتون

جدول 19. در یک نگاه